Week 12: Making Choices

Ellen Bledsoe

Making Choices

Conditionals

When we want to make choices in R, we are most often using some type of conditional statement.

We talked about conditional statements early in the class when we talked about conditional sub-setting, and we've continued to use them frequently throughout the course in our `filter` functions!

Here are some examples below.

```
weight > 50
species == "DM"
```

When we run these lines of code, these statements produce an output that is a `"logical"` data type.

- `TRUE` if the condition is satisfied
- `FALSE` if the condition is not satisfied

As a reminder, these are not the same as strings "TRUE" and "FALSE." These are a special type of value called "boolean."

In some ways, they are similar to `NA` values. True `NA` values do not go in quotation marks because they are a special data type; the same is true for `TRUE` (or `T`) and `FALSE` (`F`).

Conditional Operators

As we've seen in previous classes, conditional statements can be created with a number of different operators.

We have the "traditional" operators:

- `==` for equals
- `!=` for not equals
- `<` and `>` for less than or greater than
- `<=`, `>=` for less than or equal to or greater than or equal to

There are other operators that we have not yet used, such as `%in%`. This is a very handy little operator!

It checks to see if a value is present in a vector of possible values.

```
"DM" %in% c("DM", "DO", "DS")
```

```
[1] TRUE
```

```
"PP" %in% c("DM", "DO", "DS")
```

```
[1] FALSE
```

We can also combine conditional statements using `&` (and) and `|` (or). With `&`, both statements need to be `TRUE`; with `|`, only one of them does.

```
x <- 7
x > 5 & x < 10
```

```
[1] TRUE
```

```
x < 5 | x > 6
```

```
[1] TRUE
```

There are also a number of functions that return these boolean data types.

For example, the `is.na()` function returns `TRUE` or `FALSE` depending on whether a value is null or not.

```
is.na(NA)
```

```
[1] TRUE
```

```
is.na(5)
```

```
[1] FALSE
```

Reversing Conditions

When we want the opposite of a certain condition, we can use the `!` operator to negate the condition.

In doing so, it reverses the boolean values that are returned (`TRUE` becomes `FALSE` and `FALSE` becomes `TRUE`).

```
!(10 >= 5)
```

```
[1] FALSE
```

```
!is.na(c(4, NA, 76))
```

```
[1]  TRUE FALSE  TRUE
```

Notice that when we use a conditional statement on a vector, we get a `TRUE` or `FALSE` for each value in the vector. We'll come back to this later in the lesson.

> Work on Question 1: Choice Operators

Making Choices in the `tidyverse`

The `tidyverse` (`dplyr` in particular) has some very handy functions for making choices.

```
library(dplyr)
```

```
Attaching package: 'dplyr'
```

```
The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```
library(stringr)
```

Binary Choices

When the choice is binary (choose one of two outcomes), we can use the `if_else` function.

The arguments for an `if_else` function are:

- the conditional statement (`condition` argument)
- the outcome if the statement is `TRUE` (`true` argument)
- the outcome if the statement is `FALSE` (`false` argument)

```
veg_type <- "grass"

# if veg_type is grass, it is small. If not, it is large
if_else(condition = veg_type == "grass",
        true = "small",
        false = "large")
```

```
[1] "small"
```

Multiple Choices

What if we have more than two options? We can also nest the `if_else` functions.

```
veg_type <- "shrub"

if_else(veg_type == "grass", true = "small",
        false = if_else(veg_type == "shrub", true = "medium", false = "large"))
```

```
[1] "medium"
```

When we have situations where we would need to nest `if_else` statements, these nested statements can get pretty hard to interpret.

In order to avoid these nested `if_else` statements, we can use the `case_when` function.

The `case_when` function uses the `~` to set expressions.

```
veg_type <- "tree"

case_when(veg_type == "grass" ~ "small",
          veg_type == "shrub" ~ "medium",
          veg_type == "tree" ~ "large")
```

```
[1] "large"
```

`case_when` checks each condition in order and uses the first one that is `TRUE`.

When the input does not match any of the options, the default value that is returned is `NA`. In a circumstance where you want something other than this returned, you can modify the `.default` argument.

```
veg_type <- "forb"

# no match, so returns NA
case_when(veg_type == "grass" ~ "small",
          veg_type == "shrub" ~ "medium",
          veg_type == "tree" ~ "large")
```

```
[1] NA
```

```
# no match, so returns the .default value
case_when(veg_type == "grass" ~ "small",
          veg_type == "shrub" ~ "medium",
          veg_type == "tree" ~ "large",
          .default = "unknown")
```

```
[1] "unknown"
```

Using `if_else` and `case_when` with `mutate`

One of the nicest aspects of the `if_else` and `case_when` functions is that we can use them within mutate functions to make new columns (or edit existing columns) in our dataframes.

Let's look at an example using a dataframe from a package called `lterdatasampler`. This package has a number of cleaned datasets from Long Term Ecological Research (LTER) sites around the US.

```
#install.packages("lterdatasampler")
library(lterdatasampler)
```

We are going to use the `and_vertebrates` dataset, which contains data about trout and salamanders from Andrews Forest LTER.

```
and_vertebrates
```

```
# A tibble: 32,209 x 16
    year sitecode section reach  pass unitnum unittype vert_index pitnumber
   <dbl> <chr>    <chr>   <chr> <dbl>   <dbl> <chr>         <dbl>     <dbl>
 1  1987 MACKCC-L CC      L         1       1 R                 1        NA
 2  1987 MACKCC-L CC      L         1       1 R                 2        NA
 3  1987 MACKCC-L CC      L         1       1 R                 3        NA
 4  1987 MACKCC-L CC      L         1       1 R                 4        NA
 5  1987 MACKCC-L CC      L         1       1 R                 5        NA
 6  1987 MACKCC-L CC      L         1       1 R                 6        NA
 7  1987 MACKCC-L CC      L         1       1 R                 7        NA
 8  1987 MACKCC-L CC      L         1       1 R                 8        NA
 9  1987 MACKCC-L CC      L         1       1 R                 9        NA
10  1987 MACKCC-L CC      L         1       1 R                10        NA
# i 32,199 more rows
# i 7 more variables: species <chr>, length_1_mm <dbl>, length_2_mm <dbl>,
#   weight_g <dbl>, clip <chr>, sampledate <date>, notes <chr>
```

The `species` column contains the common name of each animal. Let's use `if_else` along with the `str_detect` function from `stringr` (remember that from Week 7?) to make a new column that tells us whether each animal is an amphibian or a fish.

```
and_vertebrates %>%
  mutate(group = if_else(str_detect(species, "salamander"), "amphibian", "fish"),
         .after = species)
```

```
# A tibble: 32,209 x 17
    year sitecode section reach  pass unitnum unittype vert_index pitnumber
   <dbl> <chr>    <chr>   <chr> <dbl>   <dbl> <chr>         <dbl>      <dbl>
 1  1987 MACKCC-L CC      L         1       1 R                 1         NA
 2  1987 MACKCC-L CC      L         1       1 R                 2         NA
 3  1987 MACKCC-L CC      L         1       1 R                 3         NA
 4  1987 MACKCC-L CC      L         1       1 R                 4         NA
 5  1987 MACKCC-L CC      L         1       1 R                 5         NA
 6  1987 MACKCC-L CC      L         1       1 R                 6         NA
 7  1987 MACKCC-L CC      L         1       1 R                 7         NA
 8  1987 MACKCC-L CC      L         1       1 R                 8         NA
 9  1987 MACKCC-L CC      L         1       1 R                 9         NA
10  1987 MACKCC-L CC      L         1       1 R                10         NA
# i 32,199 more rows
# i 8 more variables: species <chr>, group <chr>, length_1_mm <dbl>,
#   length_2_mm <dbl>, weight_g <dbl>, clip <chr>, sampledate <date>,
#   notes <chr>
```

The `section` column denotes whether the data were collected in clear cut forest ("CC") or old growth forest ("OG").

Let's make a new column in the data frame that has "Clear Cut" when the value is "CC", "Old Growth" when the value is "OG", and `NA` if there is something else.

```
and_vertebrates %>%
  mutate(ForestType = case_when(section == "CC" ~ "Clear Cut",
                                section == "OG" ~ "Old Growth",
                                .default = NA),
         .after = section)
```

```
# A tibble: 32,209 x 17
    year sitecode section ForestType reach  pass unitnum unittype vert_index
   <dbl> <chr>    <chr>   <chr>      <chr> <dbl>   <dbl> <chr>         <dbl>
 1  1987 MACKCC-L CC      Clear Cut  L         1       1 R                 1
 2  1987 MACKCC-L CC      Clear Cut  L         1       1 R                 2
 3  1987 MACKCC-L CC      Clear Cut  L         1       1 R                 3
 4  1987 MACKCC-L CC      Clear Cut  L         1       1 R                 4
 5  1987 MACKCC-L CC      Clear Cut  L         1       1 R                 5
 6  1987 MACKCC-L CC      Clear Cut  L         1       1 R                 6
 7  1987 MACKCC-L CC      Clear Cut  L         1       1 R                 7
 8  1987 MACKCC-L CC      Clear Cut  L         1       1 R                 8
 9  1987 MACKCC-L CC      Clear Cut  L         1       1 R                 9
```

```
10  1987 MACKCC-L CC      Clear Cut  L         1       1 R                10
# i 32,199 more rows
# i 8 more variables: pitnumber <dbl>, species <chr>, length_1_mm <dbl>,
#   length_2_mm <dbl>, weight_g <dbl>, clip <chr>, sampledate <date>,
#   notes <chr>
```

Work on Question 2: Using `dplyr` Choice Functions

Testing One Condition with an `if` Statement

So far, we've been using `if_else` and `case_when` to make choices. Remember from the beginning of the lesson that when we use a conditional statement on a vector, we get a `TRUE` or `FALSE` for each value in the vector. The `if_else` and `case_when` functions work the same way: they check the condition for each value and return a new value for each one. That is why they work so nicely inside `mutate`.

Sometimes, though, we don't want to make a choice about each value in a column. Instead, we want to choose which code to run. For example, we might want to use a different equation depending on the type of vegetation, or only run a chunk of code if a data frame has the columns we need. For these situations, we use `if` statements.

Note that the conditional statement to be tested goes inside a parenthetical. Then, whatever action is to be taken if that condition is met goes inside curly braces (`{}`), similar to how we write a function.

```
# if (conditional statement is TRUE) {
#   do something
# }
```

For example, let's say we have a value, `x`. If `x` is 5 or less, it can stay the way it is. However, if `x` is more than 5, we want to double it.

```
# create object
x <- 6

# the statement inside {} will run when the conditional statement is TRUE
# if x is more than 5, it should be doubled
if (x > 5){
  x <- x * 2
}

# check output
x
```

```
[1] 12
```

What happens when the conditional statement is `FALSE`?

```
x <- 4

# the statement inside {} will not be run when the conditional statement is FALSE
# if x is 5 or less than 5, it should stay the same
if (x > 5){
  x <- x * 2
}

x
```

```
[1] 4
```

One important difference: because an `if` statement makes a single choice, it can only check one `TRUE` or `FALSE` value at a time. Let's see what happens if we give it a whole column.

```
if (and_vertebrates$section == "CC") {
  print("Clear Cut")
}
```

```
Error in `if (and_vertebrates$section == "CC") ...`:
! the condition has length > 1
```

As a reminder, this `if` statement is *not* a function, so everything that happens in the `if` statement influences the global environment.

Let's look at another example, this time with some character values.

Perhaps different vegetation types require different biomass calculations.

```
# set objects
veg_type <- "shrub"
volume <- 16.08

if (veg_type == "shrub") {
  biomass <- 2.65 * volume^0.9
}

biomass
```

```
[1] 32.27775
```

Testing Multiple Conditions with `if-else` Statements

Often, we want to choose one of several options rather than just one.

To do this, we can add more conditions and associated actions with `else if`. We can also specify what should happen if none of the conditions are `TRUE` using `else` on its own.

```
# set objects
veg_type <- "grass"
volume <- 16.08

# if - else
if (veg_type == "shrub"){
  # if veg_type is shrub, use this equation
  biomass <- 2.65 * volume^0.9
} else {
  biomass <- NA
}



# if - else if
if (veg_type == "shrub") {
  # if veg_type is shrub, use this equation
  biomass <- 2.65 * volume^0.9
} else if (veg_type == "grass") {
  # if veg_type is grass, use this equation instead
  biomass <- 0.65 * volume^1.2
} else {
  # if veg_type is equal to anything other than shrub or grass, return NA
  biomass <- NA
}

biomass
```

```
[1] 18.21615
```

The code above checks the first condition. If `TRUE`, it will run that condition's code and skip the rest. If `FALSE`, it will check the next one and keep going until it runs out of conditions to check.

This is different from writing multiple separate `if` statements, which would check each condition separately and run the code for all of the conditions that are `TRUE`.

Work on Question 3a

Conditionals Inside Functions

So far, we've used a conditional to estimate biomass for different types of vegetation.

This seems like code we would want to reuse, so it would be a good idea to create a function to do this.

We can place our entire `if-else` statement inside a function, making sure that the function takes all required variables as arguments.

```
est_biomass <- function(volume, veg_type){
  # function to calculate biomass based on vegetation type and volume value
  if (veg_type == "shrub") {
    biomass <- 2.65 * volume^0.9
  } else if (veg_type == "grass") {
    biomass <- 0.65 * volume^1.2
  } else {
    biomass <- NA
  }
  return(biomass)
}
```

Now, we can run this function with different vegetation types and get different estimates for mass.

```
est_biomass(24, "shrub") # uses the first equation
```

```
[1] 46.28454
```

```
est_biomass(24, "grass") # uses the second equation
```

```
[1] 29.45553
```

```
est_biomass(24, "tree")  # returns NA because first two conditions were FALSE
```

```
[1] NA
```

Work on Questions 3b-3c and Question 4

Why `if-else if`?

If `dplyr` has such nice functions for making choices, why did I teach you about the seemingly more complicated `if-else if` structures?

There are a few reasons.

1) When you see other people's code, you are very likely to come across the `if-else if` structure, and you should know what it means and how to use it.
2) While `if_else` and `case_when` are really handy for somewhat straight-forward decisions, sometimes you need to perform a lot of tasks if a condition is true or false. It is much easier to add in multiple tasks to do in the `if-else if` structure than in the `tidyverse` structure.
3) The `dplyr` functions serve a very specific purpose; they are almost always used for modifying data points in data frames, because that is how the `tidyverse` is designed to work. More complicated `if-else` statements are much more flexible and can do a lot more, such as controlling your workflow. For example: (a) if a statistical test throws a specific error, make it stop running or (b) if a data frame meets one set of requirements, send it through one data cleaning pipeline, otherwise send it through a different pipeline.